🔷 Crear tabla

```
CREATE TABLE users (
id SERIAL PRIMARY KEY,
nombre TEXT NOT NULL,
email TEXT UNIQUE NOT NULL,
created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);
```

🔷 Tipos importantes

- `SERIAL` → autoincrement
- `TEXT / VARCHAR`
- `INTEGER`
- `BOOLEAN`
- `TIMESTAMP`
- `JSONB` ⭐

🧩 2. MODIFICAR ESTRUCTURA

🔷 Agregar columna

```
ALTER TABLE users ADD COLUMN edad INTEGER;
```

🔷 Eliminar columna

```
ALTER TABLE users DROP COLUMN edad;
```

🔷 Renombrar columna

```
ALTER TABLE users RENAME COLUMN nombre TO name;
```

🔐 3. CONSTRAINTS (REGLAS)

🔷 UNIQUE

```
ALTER TABLE users ADD CONSTRAINT unique_email UNIQUE (email);
```

🔷 FOREIGN KEY

```
ALTER TABLE posts
ADD CONSTRAINT fk_user
FOREIGN KEY (user_id) REFERENCES users(id);
```

🔷 CHECK

```
ALTER TABLE users ADD CONSTRAINT edad_check CHECK (edad >= 18);
```

🧪 4. CRUD (LO MÁS USADO)

🔷 INSERT

```
INSERT INTO users (nombre, email)
VALUES ('Juan', 'juan@test.com');
```

🔷 SELECT básico

```
SELECT * FROM users;
```

🔷 WHERE

```
SELECT * FROM users WHERE edad > 18;
```

🔷 UPDATE

```
UPDATE users
SET nombre = 'Pedro'
WHERE id = 1;
```

🔷 DELETE

```
DELETE FROM users WHERE id = 1;
```

🔗 5. RELACIONES Y JOINS (CLAVE)

🔷 INNER JOIN

```
SELECT u.nombre, p.title
FROM users u
INNER JOIN posts p ON p.user_id = u.id;
```

🔷 LEFT JOIN

```
SELECT u.nombre, p.title
FROM users u
LEFT JOIN posts p ON p.user_id = u.id;
```

🔷 RIGHT JOIN

```
SELECT u.nombre, p.title
FROM users u
RIGHT JOIN posts p ON p.user_id = u.id;
```

📊 6. AGREGACIONES

🔷 COUNT

```
SELECT COUNT(*) FROM users;
```

🔷 GROUP BY

```
SELECT user_id, COUNT(*)
FROM posts
GROUP BY user_id;
```

🔷 HAVING

```
SELECT user_id, COUNT()
FROM posts
GROUP BY user_id
HAVING COUNT() > 2;
```

🔍 7. CONSULTAS AVANZADAS

🔷 ORDER BY

```
SELECT * FROM users ORDER BY created_at DESC;
```

🔷 LIMIT / OFFSET

```
SELECT * FROM users LIMIT 10 OFFSET 0;
```

🔷 LIKE

```
SELECT * FROM users WHERE nombre LIKE '%pedro%';
```

🔷 IN

```
SELECT * FROM users WHERE id IN (1,2,3);
```

🔷 BETWEEN

```
SELECT * FROM users WHERE edad BETWEEN 18 AND 30;
```

🧠 8. SUBQUERIES

```
SELECT *
FROM users
```

```
WHERE id IN (
SELECT user_id FROM posts
);
```

🚀 9. CTE (WITH) — NIVEL PRO

```
WITH active_users AS (
SELECT FROM users WHERE active = true
)
SELECT FROM active_users;
```

⚡ 10. FUNCIONES ÚTILES

🔷 COALESCE

```
SELECT COALESCE(nombre, 'sin nombre') FROM users;
```

🔷 NOW

```
SELECT NOW();
```

📃 11. ÍNDICES (PERFORMANCE)

```
CREATE INDEX idx_users_email ON users(email);
```

🔄 12. TRANSACCIONES (CRÍTICO)

```
BEGIN;

UPDATE accounts SET balance = balance - 100 WHERE id = 1;
UPDATE accounts SET balance = balance + 100 WHERE id = 2;

COMMIT;
```

Si falla:

ROLLBACK;

🧬 13. JSONB (MODERNO)

INSERT INTO users (nombre, data)
VALUES ('Juan', '{"edad": 25}');

SELECT data->>'edad' FROM users;

🔍 14. EXPLAIN (OPTIMIZACIÓN)

EXPLAIN ANALYZE
SELECT * FROM users WHERE email = 'test@test.com';

🏗️ 15. VIEWS

CREATE VIEW active_users AS
SELECT * FROM users WHERE active = true;

🧠 16. FUNCIONES (AVANZADO)

CREATE FUNCTION total_users()
RETURNS INTEGER AS $$
BEGIN
RETURN (SELECT COUNT(*) FROM users);
END;

LANGUAGE plpgsql; # 📦 1. CREATE DATABASE / CREATE TABLE 👉 **Sirve para definir dónde y cómo se guardan los datos** - `CREATE DATABASE` → crea el “contenedor” de tu app - `CREATE TABLE` → define la estructura (como un esquema) 💡 En backend: > Es literalmente el “modelo de datos” de tu sistema --- # 🧩 2. ALTER TABLE 👉 **Sirve para modificar tablas ya existentes** - agregar columnas - cambiar nombres - eliminar campos 💡 En la vida real: > Cuando tu app evoluciona (ej: agregas “edad” a usuarios) --- # 🔐 3. CONSTRAINTS 👉 **Sirven para proteger la integridad de los datos** - `PRIMARY KEY` → identifica cada fila - `UNIQUE` → evita duplicados - `FOREIGN KEY` → conecta tablas - `CHECK` → valida reglas 💡 Ejemplo real: > evitar que dos usuarios tengan el

mismo email --- # 🧪 4. INSERT 👉 **Sirve para guardar datos** 💡 En backend: > cuando un usuario se registra --- # 🔍 5. SELECT 👉 **Sirve para leer datos** 💡 Es el comando MÁS importante: > el backend vive leyendo datos --- # 🎯 6. WHERE 👉 **Sirve para filtrar** WHERE id = 1 💡 Ejemplo: > buscar un usuario específico --- # 🔄 7. UPDATE 👉 **Sirve para modificar datos existentes** 💡 Ejemplo real: > cambiar contraseña, actualizar perfil --- # ❌ 8. DELETE 👉 **Sirve para eliminar datos** 💡 Ejemplo: > borrar cuenta ⚠️ cuidado: > sin WHERE = borras TODO --- # 🎯 9. JOIN (CLAVE ABSOLUTA) 👉 **Sirve para combinar tablas relacionadas** ## 🔷 INNER JOIN Solo datos que coinciden 💡 Ejemplo: > usuarios con sus cursos --- ## 🔷 LEFT JOIN Trae todo de la izquierda aunque no haya relación 💡 Ejemplo: > usuarios aunque no estén inscritos --- ## 🔷 RIGHT JOIN Lo contrario del LEFT --- # 📊 10. COUNT / GROUP BY 👉 **Sirve para agrupar y analizar datos** COUNT(*) → contar GROUP BY → agrupar 💡 Ejemplo: > cuántos cursos tiene cada usuario --- # 🔍 11. HAVING 👉 **Filtrar después de agrupar** 💡 Ejemplo: > usuarios con más de 2 cursos --- # 📈 12. ORDER BY 👉 **Ordenar resultados** 💡 Ejemplo: > mostrar usuarios más recientes primero --- # 📄 13. LIMIT / OFFSET 👉 **Paginación** 💡 Ejemplo: > páginas en una API --- # 🔎 14. LIKE / IN / BETWEEN 👉 **Filtros más avanzados** - LIKE → búsqueda por texto - IN → múltiples valores - BETWEEN → rangos 💡 Ejemplo: > buscar usuarios por nombre o edad --- # 🧠 15. SUBQUERIES 👉 **Consultas dentro de consultas** 💡 Ejemplo: > usuarios que tienen posts --- # 🚀 16. CTE (WITH) 👉 **Organizar consultas complejas** 💡 Ejemplo: > dividir lógica en pasos 👉 Más limpio que subqueries --- # ⚡ 17. ÍNDICES 👉 **Aceleran las consultas** 💡 Ejemplo: > buscar por email rápido ⚠️ pero: > demasiados índices = lento al insertar --- # 🔄 18. TRANSACCIONES (MUY IMPORTANTE) 👉 **Agrupar operaciones como una sola** 💡 Ejemplo real: > transferencias de dinero - todo se ejecuta o nada --- # 🧬 19. JSONB 👉 **Guardar datos flexibles** 💡 Ejemplo: > configuraciones, metadata --- # 🔍 20. EXPLAIN ANALYZE 👉 **Ver cómo PostgreSQL ejecuta tu query** 💡 Sirve para: - optimizar - detectar problemas --- # 🏗️ 21. VIEWS 👉 **Guardar consultas como tablas virtuales** 💡 Ejemplo: > “usuarios activos” --- # 🧠 22. FUNCTIONS 👉 **Lógica dentro de la base de datos** 💡 Ejemplo: > cálculos complejos